Oscar Alvarado

I320D: Open Source Software Development

Professor James Howison

March 22, 2026

Creating and Analyze Analogies to understand Git and Github

Git and Github are both powerful resources that are commonly used to collaborate with others with ongoing projects. But in order to utilize these resources, one must understand where they are coming from and how each step in the process can be interpreted. To understand how Git and Github works, we will be using different analogies to explain. The first analogy will be from an online source, the second will be made by me, and the third will be developed using GenAI. Through these different analogies, students will be able to understand how Git and Github are used in real life to develop, alter, and copy projects.

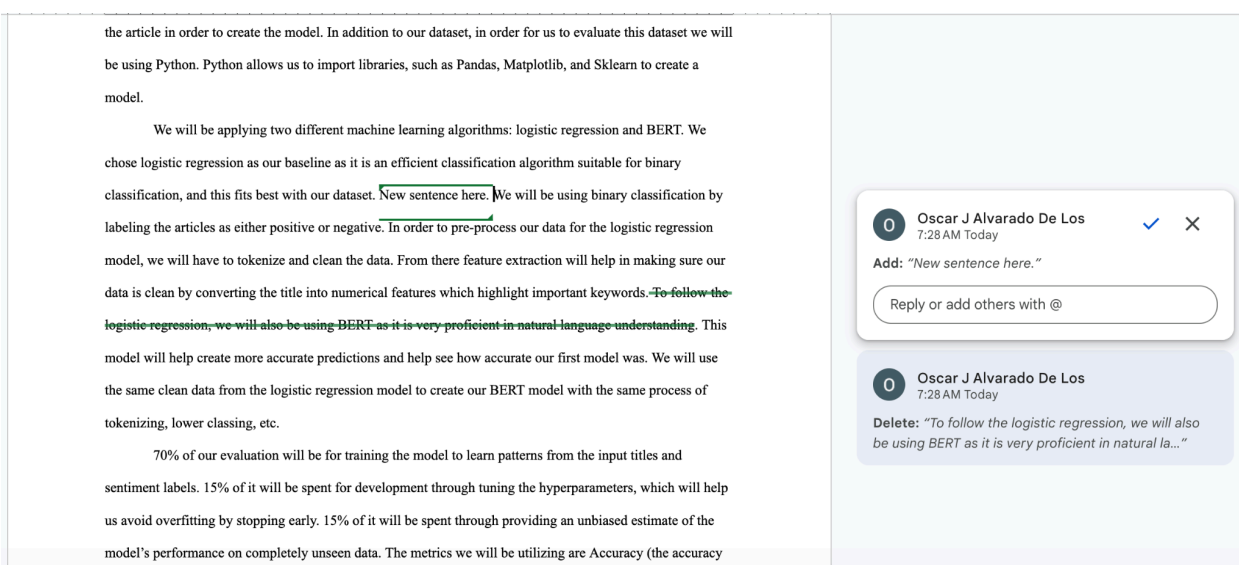
For the first analogy, I will be using an example from Medium where Iza Stan writes an analogy of a git collaboration using a cookbook. The git repository is described as a cookbook used to store important recipes with secret ingredients and memories (Stan). A git repository stores all files for a project which can be altered. It keeps track of changes throughout the project. If you were to make a copy for yourself of the cookbook, that would be considered cloning in git as you would be making a copy of the cookbook – or repository. This copy would be able to be edited without changing the original version. It's as if you were making a copy of the cookbook as it is now to keep for yourself and do what you want with it without altering the original. In order to update the original cookbook, let's say you have a new recipe, you would have to commit. Committing in Github is where new information is being added to the repository. Using Git, you are able to commit changes to the repository which is like adding a new recipe to the cookbook. When you commit something, the time and changes are uploaded to Github where they can be tracked and reviewed. Now lets say we want to experiment and change some of the recipes but don't want to change the original recipes as we aren't sure if it's going to be a permanent change. To do this, we create a new branch where the cookbook can be edited without actually changing the original. A

branch creates a copy of the repository which can be altered without changing the original. In this branch, you can change the items in the cookbook/repository to whatever you'd like. This is a great way to include new ideas that aren't ready for inclusion in the main cookbook/repository. If we decide we do want to change the cookbook to the branch we created, we create a pull request where the main owner of the cookbook can decide to amend the cookbook. A pull request will prompt the owner to accept or decline the changes made. The owner is also able to leave comments on why or why not a certain pull request is

accepted. This analogy isn't in the article but lets say the owner of the cookbook likes only a few of the changes to the recipe, in this case, the owner can split the pull request and select which commits to actually change in the cookbook/repository. Now if someone wanted to have their own copy of the cookbook to make their own and add more to their liking, in Git terms this would be forking. Through forking, you have access to the same repository but none of your changes to it will affect the original repository as the repository that is forked is owned by the person who forked it. This analogy goes over most Git concepts and helps provide an insightful way of interpreting Github.

For the second analogy, we will be using a Google document as an example. The document will be the repository where a lot of important information is stored. If we were to create a copy of the main document, that would be cloning as we keep a copy for changes in our local computer. If someone else were to want a copy, that would be considered forking. Forking allows for edits to be made to a new

version of the main document without changing the actual main document. Github is an open source platform so forking is an easy practice which can be done by anyone. When you fork a repository, you get access to all the information in the repository where you can change the repository to your liking without affecting the original document. So when we want to create a change to a repository, we will have to commit the changes. An important command used before committing is “git add” where you choose the file you want to be committed in the future. Once you have finished adding all your files is when you are able to commit. Committing helps keep track of changes to a repository, kind of similar to where we can track back to different versions of a document by looking at previous saves of the document. Though Google Docs automatically saves, pretend it didn’t and you had to save the document yourself. This is very similar to committing as committing helps you keep track of progress and can be traced back like a Google Doc. Now let’s say that we want to create some alterations to the new document but we aren’t the main owner of the repository. In order for these changes to be permanent, we have to create a pull request but need to remember to use “git push” beforehand. Git push pushes all the commits from the branch created to the repository. After doing that, we are able to create a pull request. A pull request gives the owner of the repository the opportunity to review the changes made and decide if he wants to include these changes. In terms of a Google Doc, a pull request is as if we were on a shared document in



“Suggestion” mode. The owner can see the suggestions we include but can choose whether or not to

implement these changes. If the owner of the document sees these changes but only wants to implement a few, not all. This can be referred to splitting a pull request where the owner only selects a few of the changes requested. Cherry picking is a tool to split a pull request to create a new branch selecting the desired commits and merging that branch. In our Google Doc, it's as only selecting a few of the suggestions made. A Google Doc is an interpretable way of understanding how Github and Git commands work to create a project as a Google Doc is a great tool used to share and collaborate work with others.

The following analogy was created using GenAI. To create this analogy, my first prompt was "Create an analogy that can be explained using real life examples for creating pull requests on Github?" GenAI created an analogy that compares creating a pull request to "submitting a professional architectural blueprint for a renovation." The first analogy given is the branch which is described as the drafting room. The drafting room is like a branch because the drafting room is where a copy of the original blueprints are able to be edited without actually changing the main blueprint copy and the branch does a very similar thing in git where it creates a different space where the original repository can be edited without actually changing the original repository. The real life example is wanting to add a balcony to your house. In order to do that, you make a copy of the blueprint and work from there. Next, committing is the progress photos. The progress photos show the placement in the procedure as committing does for git as you can see previous commits or "photos" of different changes made throughout the project. Below is the real life example GenAi provided for this analogy. I included a photo of it as I think it did a good job explaining it. A pull request is the formal proposal. Before actually

- **Real-life example:** First, you finish the floor plan (Save). Then, you decide on the materials for the railing (Save). Then, you calculate the weight-bearing load (Save). Each "save" is a snapshot of your work at that moment.

building your plans, you need to make sure to get them checked. You take them to the manager to review the new blueprints where he can review and check them and if they are approved, they can then be implemented into the main blueprint. This is like submitting a pull request as it is the process of getting worked checked before approval of implementation. The feedback loop is the review step according to

GenAI. As the project manager looks at the blueprints, there may be some specific issues that need to be addressed which is what happens in the review phase of a git collaboration project as you can also send comments on pull requests and ask for adjustments. Finally, GenAI uses the construction as the analogy

- **Real-life example:** The Architect says, "The balcony looks great, but those glass railings won't pass local building codes. Change them to steel." You go back to your drafting room, update the drawings, and the PR updates automatically.

for merging. Once blueprints are approved, the plans can proceed and the original blueprint can be altered to the new version. When you finish making adjustments and taking back feedback from your pull requests, your changes can then be merged into the main branch. At the end of the response given by GenAI, it had a really good suggestion which led to my follow up prompt to be “Yes, explain how "Merge Conflicts" and any others can fit into this renovation analogy?” For merge conflicts, the title of the analogy is “Two people, One wall”. In this scenario, two people are trying to renovate the same wall but have different ideas. This shows merge conflicts as this can happen on a git collaboration project where two pull requests will cause merge conflicts as they contradict each other. The solution for both of these is to collaborate and discuss how this can be solutionized or changed. Next, “The ‘What if’ House” represents forking. Let’s say you want to work on blueprints for a house you don’t really own. The only way for you to proceed is to buy a copy of the archived blueprints for the house and do your work there. You can do the same on git by going into another repository and forking it. There you can implement your changes without changing the original code. GenAI also produced an analogy for continuous integration and provided a preview of future material.

These analogies each tackle the git collaboration process in an understandable way. I think the best analogy fits each person differently as everyone has different experiences and familiarity with certain analogies. The first analogy does a great job in demonstrating how a repository can be amended and altered. It goes over a majority of concepts like committing, forking, pull requests, and branches. The cookbook analogy was very easy to understand as it is straightforward and has great relevance as many are familiar with following a recipe and food. This analogy did lack a little on the collaboration aspect of

git but my analogy did a better job on that. The second analogy discusses more on collaboration by using a Google Doc as a representation of a repository. I think this can be very transferable to different people as real-time collaboration on Google Docs is a typical experience that many understand. This analogy helps identify the key differences if forking and cloning as they can be confusing as they are very similar. I also really liked how committing was explained by the log of previous saves of a document. Since I created this analogy, I tried to use as many commands as possible which I think works pretty well and can be explained easily. For example, I also included how we need to use “git add” before a commit and “git push” before creating a pull request. This analogy does go further in depth and also discusses splitting pull requests and cherry picking. The final analogy developed using GenAI used an architectural project and had the main blueprint represent the repository. Using GenAi was useful as it developed a good analogy and also allowed for me to learn further. It would probably be the most useful as you can also use the GenAI to clarify any confusions. I really liked how forking was explained and showed the open sourceness of Github as the example used a house that wasn’t owned by them which is something you can also do in Github. Branching was explained very well in this analogy as well as pull requests by showing an emphasis on needing to be approved. Merging was a new concept discussed that is also an important term to know for Github. I also really liked that I was able to go further into my understanding and also learn a new analogy for merge conflicts which is a common encounter in Github. To get a better understanding of Git and Github, these three analogies should be analyzed and interpreted. Each of them contains overall the main focus but each has its own priorities and explains certain concepts more in depth than others. Not all examples could have deep resemblance to all concepts for a git collaboration project which is why three different analogies are helpful in understanding the variety and different concepts of a git collaboration project. I don’t think that using a combination of these analogies would be difficult as they can all be interpreted from the same concept as each contains an example for a repository, committing, branches, and pull requests. Each analogy just explains different concepts that can be applied to it as well for further use of the analogy and understanding terms. Like the second analogy uses git commands as well and the third analogy explains merge conflicts which can’t be done as easily in the first

two analogies. All analogies each hold a great amount of information and to get the best understanding should all be read as they each incorporate different features and in-depth examples.

References

Stan, Iza. "Using Git Is Like Writing a Cookbook." Medium, 28 Aug. 2018, https://medium.com/@izab.sta/using-git-is-like-writing-a-cookbook-baea19b47d61.